RELAY

Smart MCP Orchestration Layer

Product Requirements Document

The AI layer between humans and their tools.

Plug in any MCP. Speak naturally. Relay figures out the rest.

Field	Value
Document Version	1.0
Status	Draft — Active Development
Owner	Pramod
Repository	red-hat-community-ai-tools/relay
Last Updated	February 18, 2026
Classification	Internal — Open Source

1. Product Overview

1.1 The Problem

The MCP ecosystem is growing rapidly, but the developer experience is fragmented. When a developer has multiple MCPs available — GitHub, Jira, a database tool, a filesystem tool — they are forced to:

- Learn a different command vocabulary for every MCP they add
- Manually switch between MCP contexts when their real task spans multiple tools
- Understand the internals of each MCP to get meaningful work done
- Lose productivity every time they add or remove an MCP from their environment

> *The user is forced to think in tools. They should be thinking in outcomes.*

1.2 The Solution

Relay is a smart orchestration layer that sits between the developer and their MCP ecosystem. It exposes a single, stable natural language interface. Whatever MCPs are plugged in underneath, the user's interaction model never changes.

Relay handles three fundamental jobs:

1. Learn — When an MCP connects, Relay introspects it, understands its capabilities semantically, and announces what is now possible in plain English.
2. Route — When the user speaks, Relay identifies the true intent and maps it across whichever MCPs are needed to fulfill it.
3. Execute — Relay runs the plan, manages cross-MCP data flow, handles errors, and narrates the result back in plain language.

1.3 Core Design Principle

> *"The user speaks human. Relay speaks MCP." Relay is not a chatbot. It is an execution engine. It acts, reports, and moves on.*

1.4 Strategic Value

Relay is differentiated from simply loading multiple MCPs into an AI context in three critical ways:

Capability	Relay vs Raw Multi-MCP
Interaction model	Single stable interface vs per-MCP command vocabulary
Cross-tool workflows	Automated DAG execution vs manual copy-paste between tools
Dynamic capability	Plug/unplug with instant re-mapping vs manual reconfiguration
Context persistence	Session state carries across all tool calls vs stateless per-call
Failure handling	Structured recovery and narration vs raw error passthrough
Discoverability	Natural language announcement of new capabilities vs reading docs

2. System Architecture

2.1 High-Level Design

Relay is structured as four cooperating subsystems. Each has a single, clear responsibility.

```
                    ┌──────────────────────────────────┐
                    │          USER (Natural Language)          │
                    └──────────────────────────────────┘
                                   │
                                   ▼
                    ┌──────────────────────────────────┐
                    │              RELAY CORE                   │
                    │                                           │
                    │   ┌──────────┐  ┌──────────┐              │
                    │   │ Registry │  │  Learner │              │
                    │   └──────────┘  └──────────┘              │
                    │   ┌──────────┐  ┌──────────┐              │
                    │   │  Router  │  │ Executor │              │
                    │   └──────────┘  └──────────┘              │
                    └──────┬──────────────┬──────────────┘
                           │              │
              ┌────────────┘              └────────────┐
              ▼            ▼                ▼            ▼
          [GitHub]      [Jira]          [Postgres]    [Slack]
            MCP          MCP               MCP          MCP
                  ← plug / unplug at any time →
```

2.2 Subsystem Responsibilities

Registry — Source of Truth

Maintains the live index of all plugged-in MCPs, their tools, and the semantic capability map that the Router queries. When an MCP plugs in, the Registry connects, triggers the Learner, and stores the result. When an MCP unplugs, the Registry cleanly removes all associated entries from the capability map.

Learner — Capability Intelligence

Activated on every plug event. Takes raw tool schemas from the MCP and uses an LLM to enrich each tool with: the human intent it serves, the action verbs that map to it, example phrases a user might say, what the tool returns in plain English, and its risk level (safe, reversible, or destructive). Produces the announcement text that is shown to the user when the MCP connects.

Router — Intent Engine

Receives every user message. Searches the capability map for candidate tools using semantic matching. Invokes the LLM to determine workflow type: single-tool, sequential multi-tool, parallel multi-tool, ambiguous, or out-of-capability. Produces an execution plan or a single clarifying question. Never produces more than one question. Prefers acting over asking.

Executor — Plan Runner

Receives the plan from the Router and runs it step by step. Resolves inputs across steps by semantic field matching, not field name matching. Handles destructive step confirmation. Returns a structured result — success with all step outputs, or failure with the exact step and error — which is passed to the Narrate mode of the LLM for user-facing response.

2.3 The LLM Prompt Architecture

Relay uses a single master prompt with four operating modes. Each user interaction involves one or more LLM calls, each in a specific mode. The prompt accepts runtime parameters injected at call time.

Mode	Triggered When	LLM Job
LEARN	MCP plugs in	Enrich tools semantically, generate announcement
ROUTE	User sends any message	Parse intent, identify workflow type, decide if plan proceeds
PLAN	Route says proceed	Build step-by-step DAG with MCP + tool + inputs per step
NARRATE	Execution completes	Translate raw output into plain English response

2.4 Runtime Parameters

Every LLM call requires these runtime values injected into the prompt. This is the contract between the application code and the LLM:

Parameter	Description
active_mode	One of: learn | route | plan | narrate
plugged_in_mcps	Serialized list of connected MCPs with their tool schemas
capability_map	Current intent keyword → tool mapping (route mode)
session_context	Prior messages and tool outputs for this session
user_message	The raw user input (route, plan, narrate modes)
raw_tools	Raw tool list from MCP introspection (learn mode only)
route_result	Output from ROUTE call (plan mode input)

execution_result	Step outputs from Executor (narrate mode input)

3. Functional Requirements

3.1 Plug / Unplug Lifecycle

4. When a user runs relay plug <name>, Relay establishes an MCP connection using the provided configuration.
5. Relay calls list_tools() on the MCP to retrieve all available tool schemas.
6. The Learner processes each tool through the LLM in LEARN mode and produces enriched metadata.
7. The Registry indexes enriched tools into the capability map by intent keywords and action verbs.
8. Relay prints an announcement to the user in plain English describing what is now possible.
9. When a user runs relay unplug <name>, the MCP is disconnected and all associated capability map entries are removed.
10. After unplug, Relay confirms which capabilities are no longer available.

3.2 Natural Language Routing

11. Every user message is processed through ROUTE mode regardless of what MCPs are plugged in.
12. The Router searches the capability map using semantic intent matching, not keyword-only search.
13. If no plugged-in MCP can handle the request, Relay says so clearly and suggests which MCP would help.
14. If the intent is ambiguous, Relay asks exactly one clarifying question and waits.
15. Relay never exposes MCP tool names, raw schemas, or JSON to the user.

3.3 Cross-MCP Execution

16. The Executor runs steps in the order defined by the plan DAG.
17. Outputs from one step are passed to dependent steps by semantic field matching.
18. For destructive steps, Relay shows a confirmation summary and requires explicit approval before proceeding.
19. If a step fails, Relay reports the specific failure in plain English and suggests a concrete next action.
20. Relay never silently continues past a failed step.

3.4 Session Context

21. Relay maintains session context across all messages and tool calls within a session.
22. Context includes prior user messages, inferred working state (e.g., active branch, active ticket), and prior tool outputs.
23. The Router uses session context to infer missing information before asking the user.
24. Context is not persisted between sessions unless the user explicitly enables persistence.

3.5 Safety and Confirmation

Risk Level	Relay Behavior
safe	Execute immediately, mention in narration what was done
reversible	Execute immediately, clearly state what happened and how to undo
destructive	Show confirmation summary, require explicit y/yes before proceeding

4. The Master Prompt Specification

4.1 Relay Identity Directive

The following identity block is fixed and never changes between invocations. It establishes what Relay is and is not:

> *You are Relay — an intelligent MCP orchestration engine. You are NOT a chatbot. You are an EXECUTION ENGINE that understands what the user wants, figures out which tools to use across plugged-in MCPs, builds a plan, executes it, and reports back in plain English. You have no personality beyond being fast, precise, and reliable. You never say 'I would be happy to help.' You act. You report. You move on.*

4.2 Runtime Injection Template

Every LLM call fills these placeholders before sending:

```
PLUGGED-IN MCPs:
{plugged_in_mcps}

CAPABILITY MAP:
{capability_map}

SESSION CONTEXT:
{session_context}

CURRENT USER INTENT:
{user_message}

ACTIVE MODE: {active_mode}
```

4.3 Mode Output Schemas

LEARN Mode Output

```
{
  "mode": "learn",
  "mcp_name": "string",
  "enriched_tools": [
    {
      "tool_id": "mcp_name:tool_name",
      "human_intent": "string",
      "action_verbs": ["string"],
      "example_phrases": ["string"],
      "output_description": "string",
```

```
      "risk_level": "safe | reversible | destructive"
    }
  ],
  "capability_summary": "string",
  "announcement": "string"
}
```

ROUTE Mode Output

```
{
  "mode": "route",
  "intent_understood": "string",
  "workflow_type": "single | sequential | parallel | ambiguous | no_capability",
  "clarification_needed": "string | null",
  "no_capability_message": "string | null",
  "suggested_mcp_to_plug": "string | null",
  "proceed_to_plan": true | false
}
```

PLAN Mode Output

```
{
  "mode": "plan",
  "needs_confirmation": true | false,
  "confirmation_message": "string | null",
  "steps": [
    {
      "step": 1,
      "mcp": "string",
      "tool": "string",
      "inputs": {},
      "input_sources": { "field_name": "user_message | session_context |
step_N_output" },
      "depends_on": null | [1, 2],
      "reason": "string",
      "risk_level": "safe | reversible | destructive"
    }
  ]
}
```

NARRATE Mode Output

```
// Success
{
  "mode": "narrate",
  "status": "success",
  "summary": "string",
  "detail": "string | null",
```

```
  "follow_up_suggestions": ["string"]
}

// Failure
{
  "mode": "narrate",
  "status": "failed",
  "failed_step": 1,
  "what_happened": "string",
  "what_to_try": "string"
}
```

5. Relay Behavioral Rules

5.1 What Relay Never Does

- Expose MCP tool names, raw schemas, or JSON blobs to the user
- Say 'I cannot' without specifying what would make the request possible
- Ask more than one question at a time
- Execute a destructive action without explicit user confirmation
- Hallucinate tool capabilities — only what is in the capability map is valid
- Break character — Relay is always the execution engine, never a general assistant
- Dump stack traces or raw errors at the user
- Continue silently past a failed step

5.2 What Relay Always Does

- Lead with action, follow with explanation
- Use session context to infer missing information before asking
- Bridge tool outputs across MCPs by semantic meaning, not field name
- Announce new capabilities in plain English when an MCP plugs in
- Suggest 1-2 natural next actions after every completed workflow
- Treat the user's words as intent, not as literal commands
- Respond in the same language register the user uses

5.3 Cross-MCP Intelligence Rules

Output Bridging

When step N output feeds step N+1 across different MCPs, Relay identifies connecting fields by semantic meaning. For example, GitHub's pull_request.head.sha and a CI tool's commit_hash are the same thing. Relay bridges them without user involvement.

Conflict Resolution

When two plugged-in MCPs can both handle a request, Relay prefers the one used more recently in session context. If equally recent, Relay prefers the one with the more specific capability match. Relay never asks the user to pick unless truly ambiguous.

Capability Gaps

When no plugged-in MCP can fulfill the request, Relay does not approximate with wrong tools. It states clearly what is missing and names the MCP that would enable the request.

6. Example Cross-MCP Workflows

6.1 GitHub + Jira: Fix and Track

User says: "fix the auth bug and close the ticket"

```
ROUTE:  workflow_type = "sequential"
        intent = "fix bug in code, commit, and close associated issue"

PLAN:   Step 1 — [jira] get_issue       inputs: { query: "auth bug" }
        Step 2 — [github] create_branch  inputs: { name: from step 1 issue key }
        Step 3 — [github] commit         inputs: { branch: step 2 output }
        Step 4 — [github] create_pr      inputs: { branch: step 2, title: step 1
summary }
        Step 5 — [jira] transition_issue inputs: { id: step 1, status: "Done", pr:
step 4 }

NARRATE: "Done. Branch fix/AUTH-142 created, PR #87 opened, and AUTH-142 closed.
          Next: assign a reviewer or add labels to the PR."
```

6.2 Filesystem + Git + Terminal: Safe Refactor

User says: "refactor the auth module"

```
ROUTE:  workflow_type = "sequential"
        intent = "refactor code files, run tests, only commit if passing"

PLAN:   Step 1 — [filesystem] read_file    inputs: { path: "src/auth" }
        Step 2 — [code] refactor            inputs: { content: step 1 output }
        Step 3 — [filesystem] write_file    inputs: { content: step 2 output }
        Step 4 — [terminal] run_tests       inputs: { scope: "auth" }   risk:
reversible
        Step 5 — [git] commit               inputs: { message: "refactor: auth
module" }
                                             depends_on: [step 4 success]

NARRATE: "Refactoring complete. 3 files updated, all 24 tests passing. Committed as
          a8f3c12. Next: push to remote or open a PR."
```

6.3 Plug Announcement Example

User runs: relay plug github

```
LEARN output → announcement:
```

```
"GitHub is connected. I can now manage your repositories, branches, pull requests,
commits, and issues. Try:
  - 'show my open PRs'
  - 'create a branch for the login bug'
  - 'what changed in main this week'"
```

7. Implementation Plan

7.1 Build Order

Build in this sequence. Each phase produces a testable, usable artifact before the next begins.

Phase	Deliverable	Done When
1 — Registry	MCPRegistry class with plug/unplug and capability_map	Can connect to a real MCP and list tools
2 — Learner	CapabilityLearner with LLM enrichment	Announcement prints on plug with real example phrases
3 — Router	IntentRouter with ROUTE + PLAN modes	Natural language maps to correct tool and plan
4 — Executor	PlanExecutor with step DAG and error handling	Multi-step workflow completes end to end
5 — Narrate	NARRATE mode wired to Executor output	User sees plain English result, never raw JSON
6 — CLI	relay plug / unplug / chat / status commands	Full session usable from terminal

7.2 First Milestone Target

Wire Registry + Learner + Router to a single real MCP (GitHub or filesystem). The test is simple: plug in the MCP, see the announcement, type a natural language command, see it execute correctly. That first working end-to-end loop proves the architecture and shows the vision.

> *When the announcement auto-generates and a natural language command routes correctly for the first time, the entire vision becomes real.*

8. Open Questions and Design Decisions

Question	Notes / Options
Persistence model	Should session context survive restarts? File-based, SQLite, or in-memory only?
MCP config format	How does the user specify connection config for relay plug? CLI flags, config file, or interactive?
LLM provider	Which LLM backs the prompt? Should this be pluggable itself?
Transparency mode	How does the user invoke 'show your work'? Flag, command, or automatic on failure?
Workflow library	When should Relay start surfacing named workflows learned from usage patterns?
Multi-user	Is Relay single-user per session or does it need a server mode for teams?
MCP authentication	How does Relay handle MCPs that require OAuth or token refresh mid-session?
Capability conflicts	Beyond recency preference, should users be able to set explicit MCP priority?

9. Glossary

Term	Definition
MCP	Model Context Protocol — a standard for exposing tools and capabilities to AI systems
Relay	The smart orchestration layer described in this document
Capability Map	Internal index mapping intent keywords to available MCP tools
Plug	The action of connecting an MCP to Relay and indexing its capabilities
Unplug	The action of disconnecting an MCP and removing its capabilities from the index
Learner	The Relay subsystem that enriches raw tool schemas with semantic metadata
Router	The Relay subsystem that maps user intent to an execution plan
Executor	The Relay subsystem that runs the plan and manages cross-MCP data flow
DAG	Directed Acyclic Graph — the step dependency structure used in execution plans

LEARN mode	LLM invocation mode for enriching a newly plugged-in MCP
ROUTE mode	LLM invocation mode for parsing user intent and determining workflow type
PLAN mode	LLM invocation mode for building the step-by-step execution plan
NARRATE mode	LLM invocation mode for translating raw results into plain English
Risk Level	Classification of a tool call as safe, reversible, or destructive
Output Bridging	Semantic matching of field values between MCP tool outputs and inputs